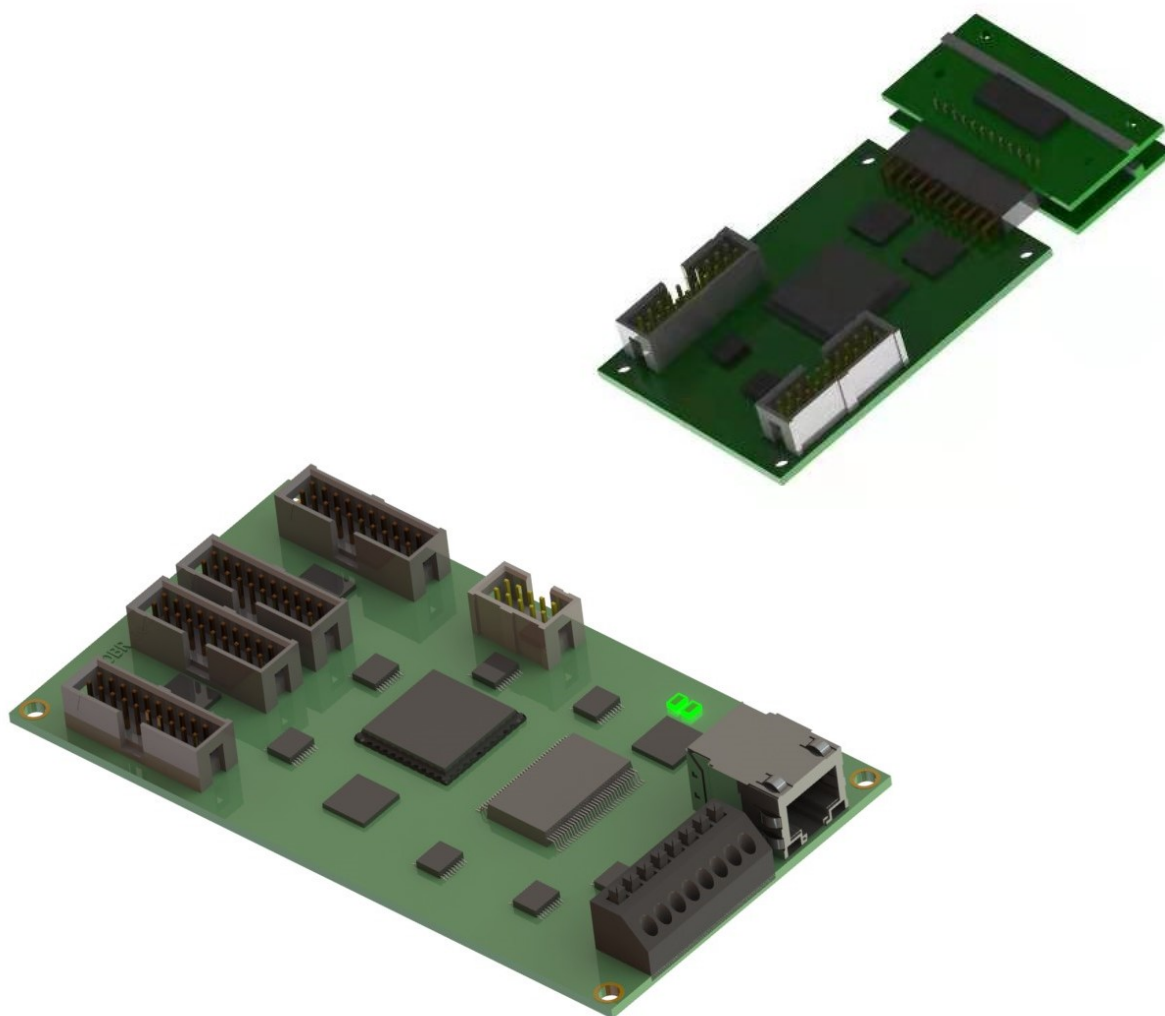


DBR&DAS 数据采集系统用户手册



版本： V1.0

第一章 用户手册简介

感谢购买锐思恒创 DBR&DAS 系列产品，DBR&DAS 数据采集系统由数据传感器单元、数据采集单元、数据传输单元以及软件平台四个核心部分组成，产品可广泛地应用于安全检查、工业无损探伤、医疗检查等领域。

1.1 物品清单

打开包装箱，检查随机文件，应包括下列文件：

- 装箱单
- 产品合格证

根据装箱单检查箱内物体，鉴于合同及客户不同要求，配件有所不同。本清单仅供参考，实际情况对照装箱单。

- D-DBR 系列数据传输主板
- D-DAS 系列数据采集板
- DS08A64 探测器
- HIGS 演示软件
- 扁平数据电缆
- 电源连接器
- 外部触发连接器

如实际物品与装箱清单存在不符或物品存在损坏，请及时与锐思恒创联系。

1.2 用途

本手册的用途在于帮助您正确的使用 DBR&DAS 系列数据采集系统。

1.3 用户手册概述

第一章 用户手册简介。

第二章 产品介绍。描述产品性能指标及工作原理。

第三章 产品安装。描述产品的硬件及软件安装。

第四章 首次运行。帮您正确实现产品的首次运行。

第五章 软件开发。帮助您实现应用程序的开发。

第六章 动态链接库 CHIGSDetector.dll 调用及开发。

第七章 常见问题及保修条款。介绍产品常见问题及解决方法，以及公司的保修条款。

附录 1 D-DBR 机械安装尺寸

附录 2 DAS 机械安装尺寸

附录 3 D-DBR 系列主板命令列表及说明

附录 4 错误代码表

附录 5 设置参数列表

第二章 产品介绍

DBR&DAS 系列数据采集系统硬件包括数据传输单元以及数据采集单元，命令及数据通过以太网协议与计算机通讯。本章描述 DBR&DAS 系列探测器参数规格及工作原理。

2.1 参数规格

表一 产品性能规格参数表

探测器类型	DS08A64DE	备注
X 射线管电压[kVp]	80-250	
闪烁体材料	HE 4mm GGAG + LE 0.5mm GGAG	注 1
像素数目	32-7680	
像素间距 [mm]	0.8	
像素高度 [mm]	0.8	
像素宽度 [mm]	0.6	
最小积分时间 [ms]	0.12	
最大积分时间 [ms]	20	
A/D 分辨率 [bit]	16	
动态范围	> 8000	注 2
数字制式 [bit]	16	
非线性度 [%]	< 0.5 %	注 3
通讯接口	RJ45 1000M	
通讯协议	UDP	
工作电压 [V]	9~24VDC	
最大功耗 [W]	DBR: 4W DS08A64DE: 2W	
工作温度 [°C]	-10—50	
相对湿度 [RH]	30—85（非结露）	
存储温度 [°C]	-15—55	

注1 可定制陶瓷 GOS 或 CsI 作为高能闪烁体。

注2 动态范围是最大响应值与本底噪声之比。

注3 非线性度为实际响应在响应范围内与理想线性曲线的最大偏离度。

2.2 工作原理

硬件系统框图如图 1 所示。

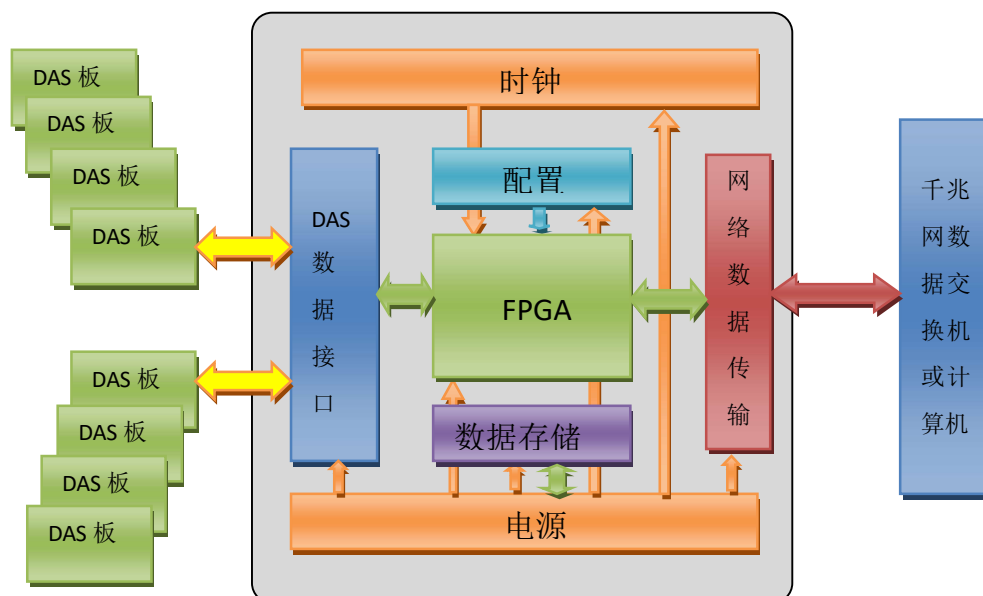


图 1 探测器数据采集系统框图

探测器原理如图 1 所示，X 射线经过探测器模块（闪烁体、光电二极管阵列及 ASIC）后，经 DAS 转换成高速数字信号，通过数据接口传送至 DBR 数据传输板，由 DBR 中 FPGA 组成的图像处理器控制图像的采集和保存，并通过千兆以太网同外部计算机通讯，使海量图像数据得以传送至计算机进行进一步图像识别处理。

2.3 D-DBR7000 接口定义

数据采集系统采用 D-DBR7000 系列主板，如图 2 所示。

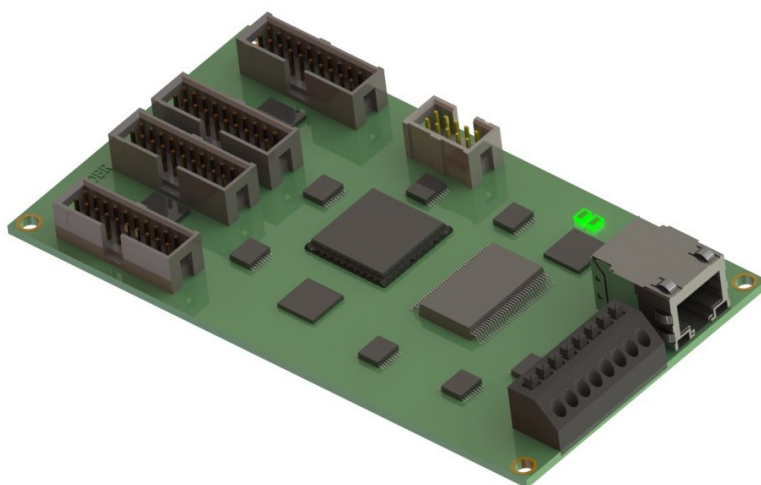


图 2 D-DBR7000 主板

表二 接口连接件型号列表

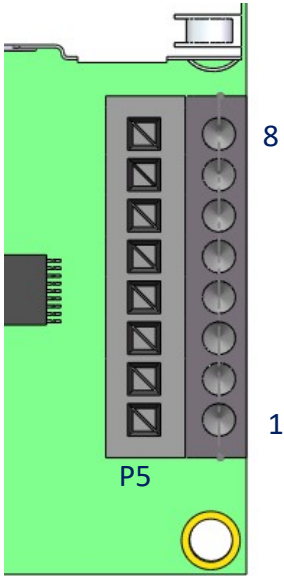
接口编码	功能描述	连接件型号
P1	信号采集接口 1	Harting9185206323
P2	信号采集接口 2	Harting9185206323
P3	信号采集接口 3	Harting9185206323
P4	信号采集接口 4	Harting9185206323
COM	串行通讯接口	WE61201021621
P5	电源接口	Molex 39700-0708
P6	RJ45 以太网接口	T 6605444-6

注：接口编码见附录 1.

2.3.1 电源接口 P5

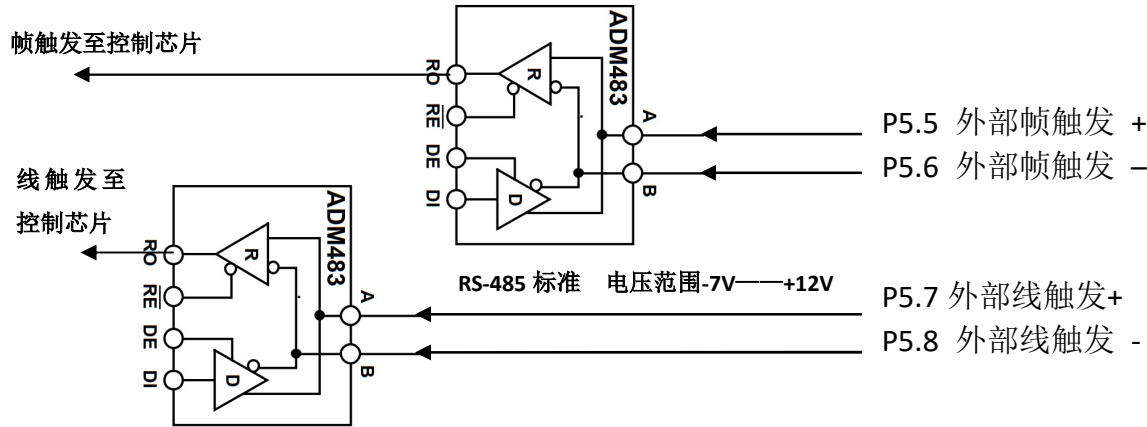
接线说明：

- P5.1 DC+ 9~24V
- P5.2 DC+ 9~24V
- P5.3 GND
- P5.4 GND
- P5.5 Frame trigger input +
- P5.6 Frame trigger common -
- P5.7 Line trigger input +
- P5.8 Line trigger common -

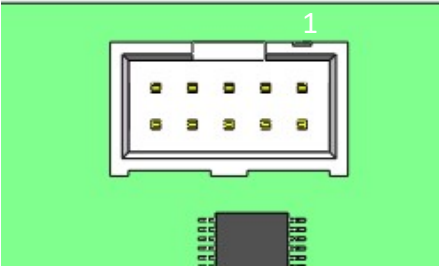


⚠ 供电接口，带电操作可能会引起器件损坏！

外部帧触发，外部线触发信号采用 RS-485 协议，外部接线参考：



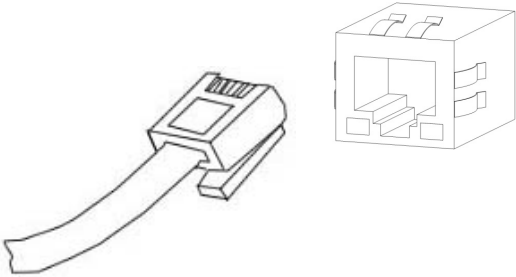
2.3.2 RS232 通讯接口 COM



接线说明：

COM.1	GND
COM.2	GND
COM.3	Input
COM.4	Input
COM.5	Output
COM.6	Output
COM.7	Rx
COM.8	Tx
COM.9	+5V
COM.10	GND

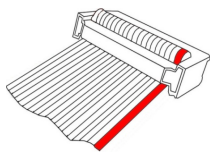
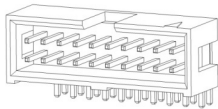
2.3.3 RJ45 以太网接口 P6



以太网电缆选用 CAT5 及以上标准线缆。

2.3.4 信号采集接口 P1, P2, P3, P4

信号采集接口数据依次排序，P1 接口数据在先，其次 P2 接口数据，再次 P3 接口数据，最后 P4 接口数据。



信号采集接口对应电缆应严格按照规范制作，否则可能导致短路。

D-DBR 具有四路信号采集通道，在一些应用中，产品仅提供两路信号采集通道（P1,P2），在连接探测器卡时遵循下列原则：

- ◆ 对称原则；
- ◆ 通道 1 优先原则。

偶数卡时，原则上平均分配于两路通道，当只选择一路通道时或者卡数为奇数时，将卡或余卡连接至通道 1。

第三章 产品安装

此安装在未通电情况下进行！



电子产品，请采取防静电措施！

本产品设计有固定孔，根据实际应用选择相应的固定孔进行固定。

3.1 D-DBR7000 数据采集板安装

D-DBR7000 数据采集板安装孔位如图所示：

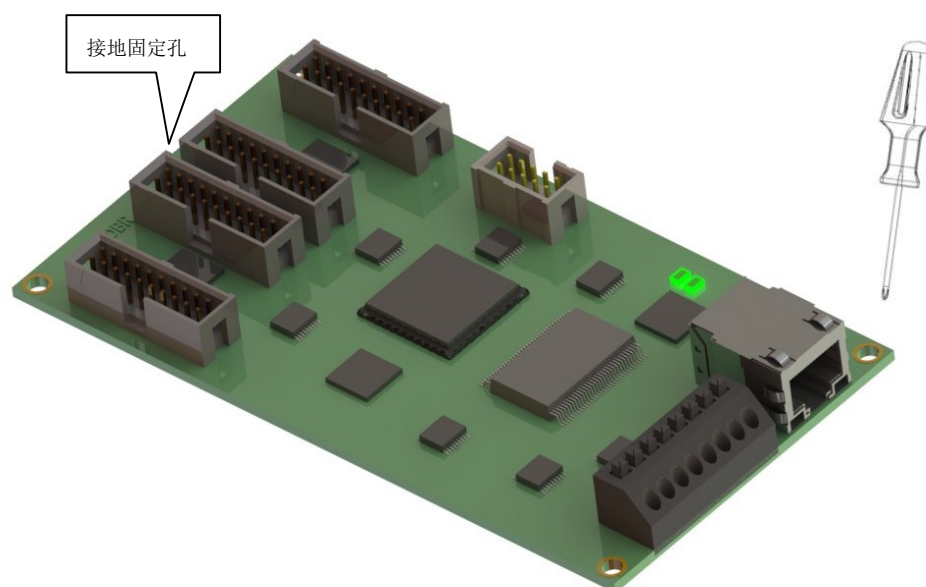


图 3 D-DBR7000 机械尺寸图



数据采集板双面装有器件，在固定中避免器件受到挤压以及避免器件与机壳接触，建议固定时采用大于 5mm 的支柱或垫片隔离。

3.2 DAS&DS08A64DE 传感器卡安装



电子产品，请采取防静电措施！

3.2.1 DAS&DS08A64DE 传感器卡介绍

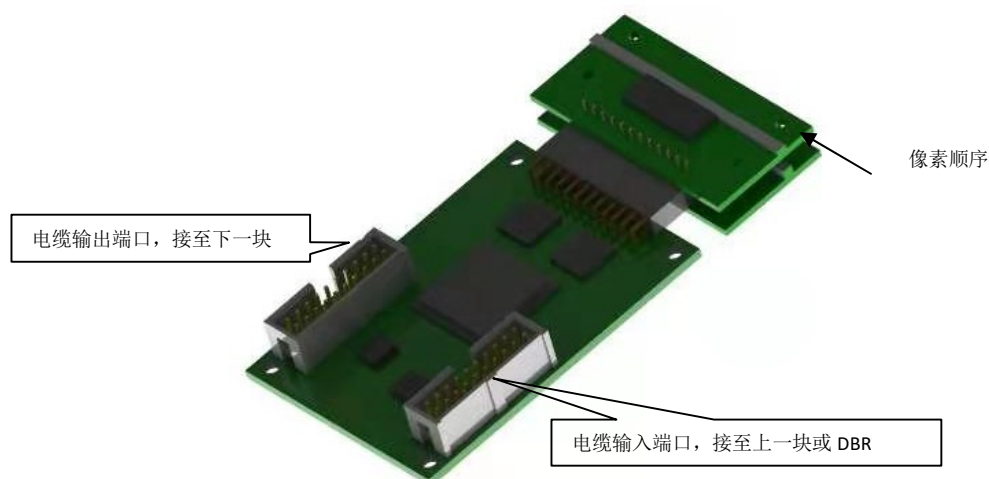


图 4 DAS&DS08A64DE 示意图

像素顺序：

如图 4 左侧箭头所示为像素顺序方向，像素所对应的灰度值数据依次输出。在设计机壳时考虑传感器卡的像素顺序，使传感器卡像素数据衔接，错误的摆放将增加电缆连接难度以及后续数据处理。

电缆输入端口：

电缆输入端口是指从主板或前一块传感器卡连接过来的电缆输入口，信号用于驱动当前传感器卡的正常工作。

电缆输出端口：

电缆输出端口是指从当前传感器输出，用于驱动下一块传感器卡的电缆输出口，作用于下一块传感器卡，如后续无传感器卡此电缆输出端口悬空。

接地固定孔：

传感器卡预留接地固定孔，根据实际情况选择接地孔的使用方式。

3.2.2 DAS 传感器卡排列

根据产品实际需求对传感器卡进行排列，可进行直线排列，也可进行 L 形排列，无论哪种排列请注意起始像素顺序以及卡与卡之间的连接关系。下图是典型的 L 形排列示意图。

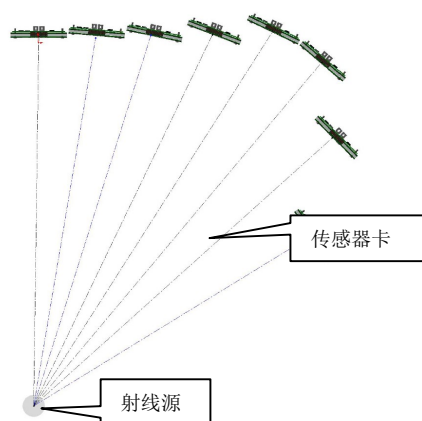


图5 L形排列示意图



传感器机械尺寸参见附录 2.

L 形排列设计中，每块传感器卡旋转一定角度，使每块卡的中心像素尽可能与射线束垂直。设计后准直机构，建议准直缝宽度为 2mm。

3.3 电缆连接

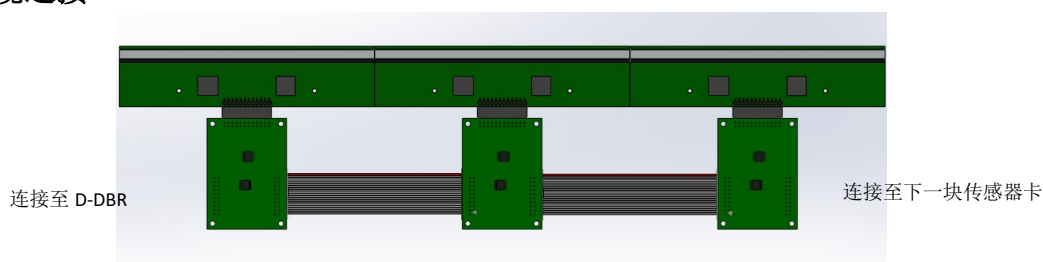


图6 线缆连接示意图



线缆中有电源信号，在制作电缆时严格保证线缆一一对应，避免出现交叉、颠倒等错误，否则可能引起主板及传感器卡的毁坏！

D-DBR7000 具有四路信号采集端口，通常当系统传感器卡数低于 8 块时，所有传感器卡可直接串接于其中一路，但每路上带的卡数量会影响系统扫描时间，建议均衡分布连接至 P1,P2,P3 以及 P4。

3.3.1 单链连接

单链连接中，建议连接 D-DBR7000 的 P1 端口，如图 7 所示。

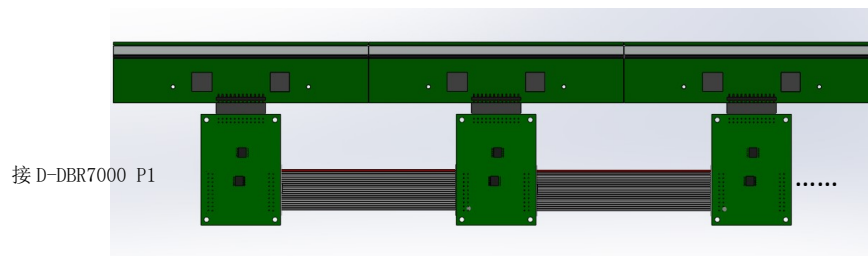


图 7 单链连接

3.3.2 多链连接

双链连接中，建议连接将传感器卡均衡连接至 D-DBR7000 的 P1，P2，P3 以及 P4 端口，如图 8 所示。

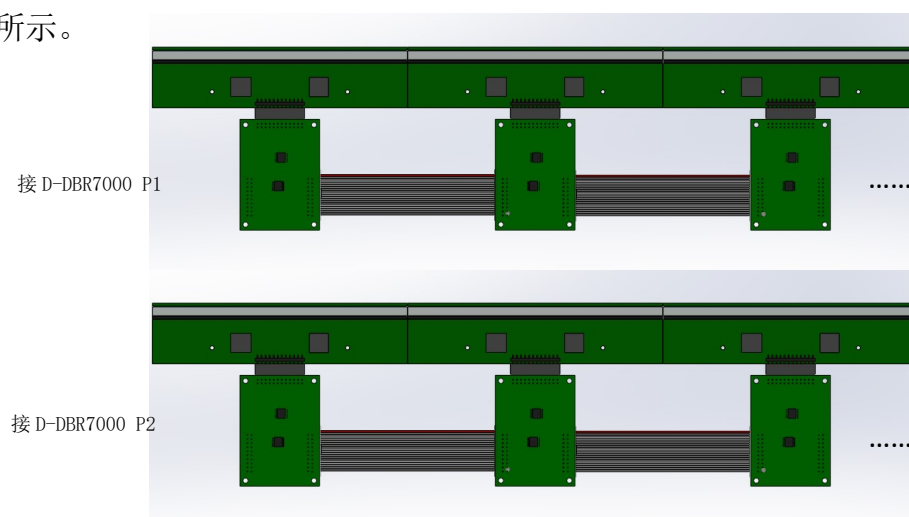


图 8 多链连接

3.4 H-Xcard demo 软件安装

计算机及系统要求：

处理器：	1G 或更高
内存：	1G 或以上
接口要求：	1000M 网络适配器
操作系统：	Windows XP (Sp2) 及以上

H-Xcard demo 安装

双击 H-Xcard demo setup.exe 进行安装，过程如下：

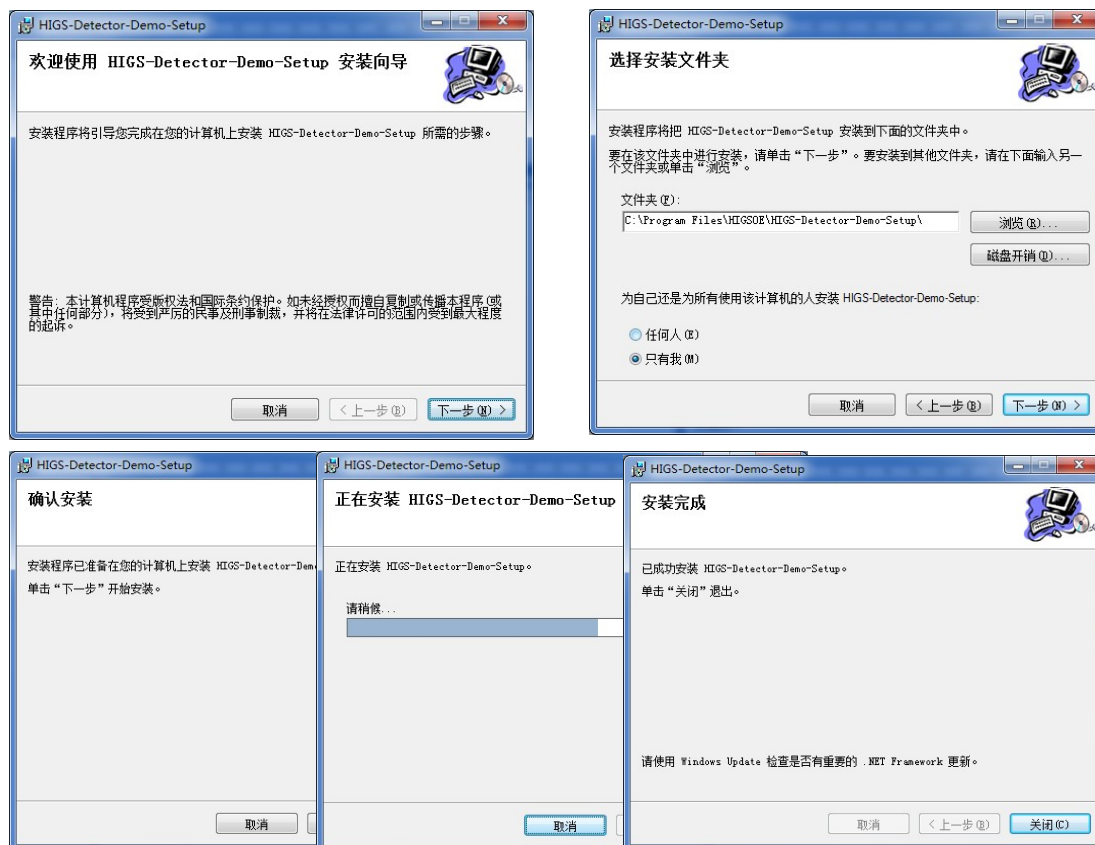


图 9 H-Xcard demo 安装过程

除了以上安装方法之外，也可以复制 Hxcard-Test Demo.exe、config.ini、orders.ini 三个文件至同一文件夹中，直接运行 Hxcard-Test Demo.exe 即可。

第四章 首次运行

D-DBR7000 及各传感器卡电缆连接完成，检查确认连接无误后，可打开电源开关对系统供电。上电后电源指示灯亮，如电源指示灯未亮，请切断供电，检查电缆及线路板。

4.1 网络防火墙设置

初次启动 H-Xcard demo 软件，操作系统（Windows7 及以上）通常会弹出如图 10 所示的窗口，这时应该勾选专用网络及公共网络，点击确定即可。

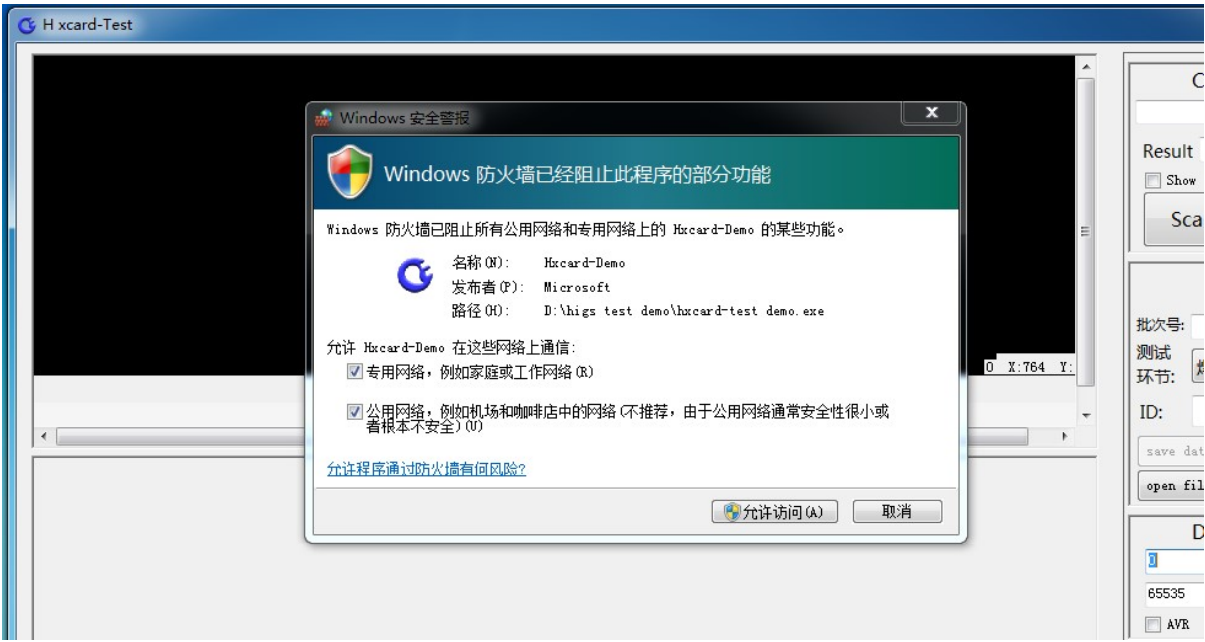


图 10

如果勾选不全，会导致软件不能使用网络或只能收发简单指令，却不能连续接收网络数据。如有以上情况发生，可以在控制面板-->Windows 防火墙-->“允许程序或功能通过 Windows 防火墙”设置中找到并选中“H-Xcard Demo”项，点击“更改设置”，然后将其对应的专用及公用两个复选框都勾选即可。

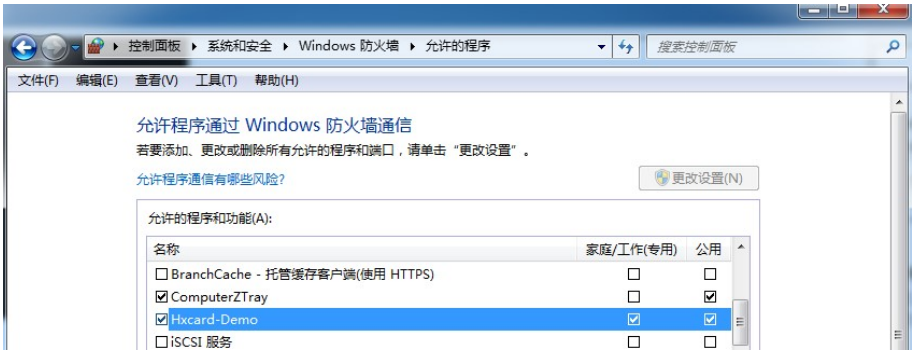


图 11

4.2 启动 H-Xcard demo 程序

双击软件图标， 软件启动。

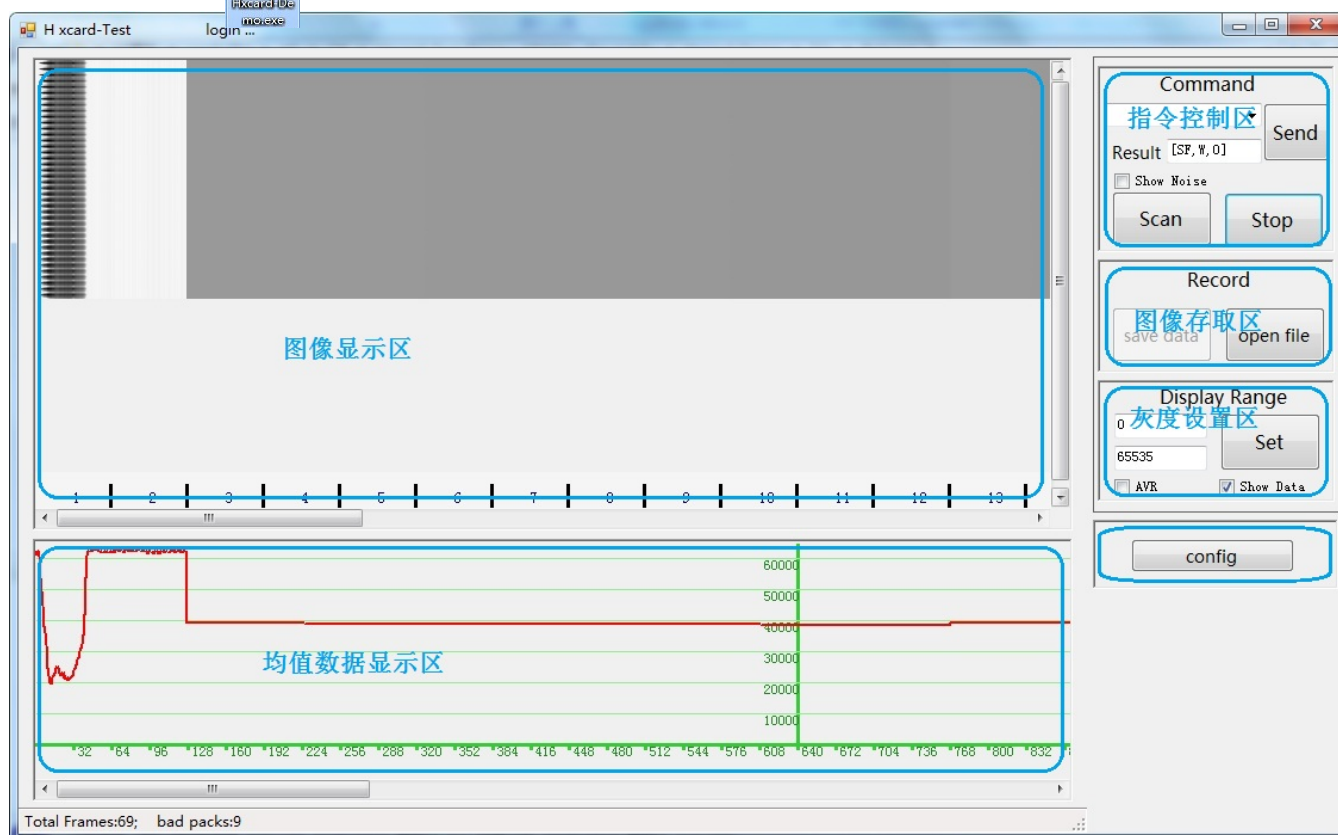


图 12 H-Xcard demo 程序界面

4.2.1 图像显示区

该区域依据指令设置动态显示检测图像。图像显示更新的时间间隔、图像的高度（显示帧数）可在配置中设置。如果图像宽度（每帧像素数大于显示区域宽度）超出显示屏的范围，拖动图像显示区下部的滚动条即可查看全部图像。

勾选“灰度设置区”下方的“Show Data”复选框，可以在鼠标悬停图像显示区上时实时显示该点坐标及灰度值。

4.2.2 指令控制区

在下拉框中输入指令（注意指令需包含在 “[]” 中，字母均为大写），也可以点击在下拉列表中选择常用命令，点击 **Send** 按钮发送；指令的返回值在 **Result** 后面显示。指令列表详见附录 3。常用指令可以编辑在 **order.ini** 文件中。

软件打开后，首先对传感器卡数进行设置，在指令控制区发送指令，例如，系统中通过 P1 连接了 8 块单能卡，通过指令窗口发送 “[DN,W,0,08]” 指令，返回 “[DN,W,0]” 后确认设置成功。

在指令窗口中发送[ST,W,***]实现积分时间设置。

点击 Scan 按钮，探测器开始扫描。

点击 Stop 按钮，探测器停止扫描。扫描停止时，可以点击“图像存取区”下部的“save data”按钮保存当前扫描结果为 txt 文件。如图 12 所示。

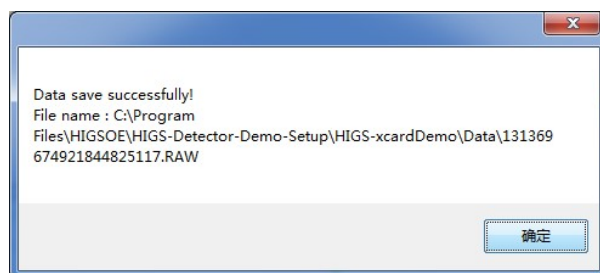


图 13 保存扫描数据

4.2.3 均值数据显示区

实时计算当前一屏数据每行像素的均值并绘制曲线图，用于辅助分析图像特征。

4.2.4 灰度设置区

灰度设置用于更改图像的显示灰度，通过设置显示灰度的最小值和最大值（0~65535），从而调整图像的亮度和对比度。复选框“AVR”选中后，均值数据显示也同灰度设置一致变化。

4.2.5 图像存取区

当停止扫描时，“save data”按钮变为可以点击状态，这时可以保存当前显示的一屏图像数据文件，文件保存在当前目录的“DATA”文件夹中。

点击“open file”按钮可以打开图像数据文件，有“txt”和“raw”两种扩展名，打开文件将显示在图像显示区中。

4.2.6 配置 “Config”

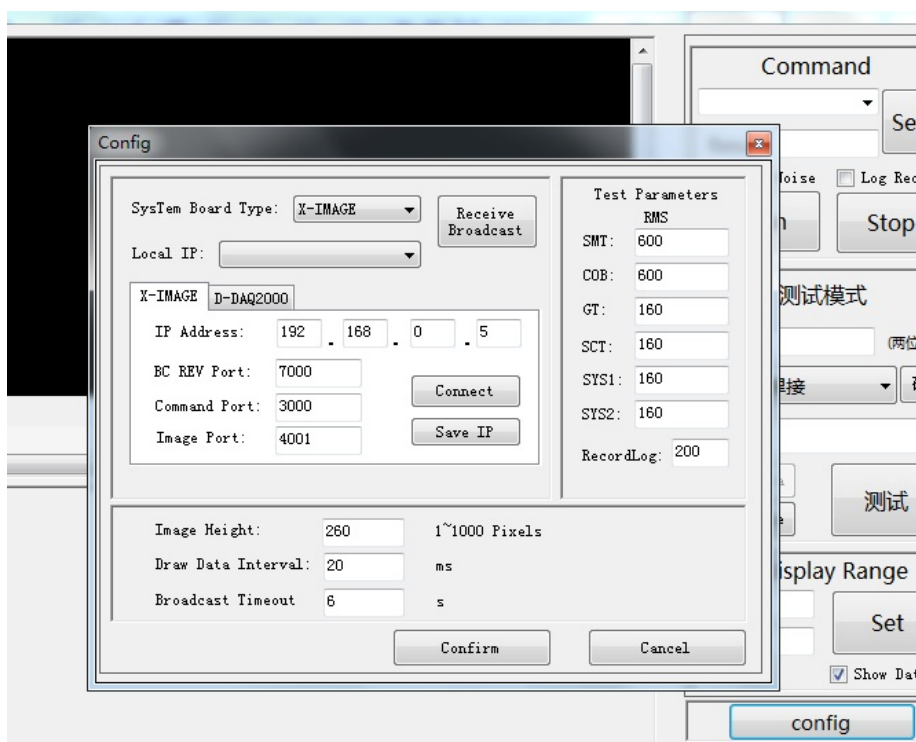


图 14

点击“灰度设置区”下方的“Config”按钮，会弹出配置窗口，在这里可以设置并保存软件及探测器的基本信息。

以初次连接 D-DBR7000 为例：

- ① System Borad Type 选择 “DBR&DAS”；
- ② Local IP 选择本机网卡的 IP 地址；
- ③ BC Rev Port 填写接收广播端口号；

④ 点击“Receive Broadcast”按钮，如显示“Detector connect successfully!”，表明连接探测器成功，可以正常使用；如显示“Connect detector failed! Correct IP address and restart software!”，说明没有正确连接到探测器，此时应该检查硬件连接、探测器供电及广播端口、本地 IP 地址是否正确。

软件安装目录下 config.ini 文件,作为配置文件，这里保存着一些缺省的参数。用户可以在这里更改“=”右侧的数值，但不能修改左侧的参数名称。如果改动导致文件读取错误，会提示“Config file read error!”，这时应删除 config.ini 文件，重启软件，会自动生成缺省的配置文件。config 文件改动后，需要重启软件使新的设置生效。

第五章 软件开发

H-Xcard demo 提供基本操作功能，您可根据实际应用开发自主应用程序，本章将介绍基于通讯协议开发客户应用程序。

5.1 广播

探测器与计算机组成服务器-客户端的网络结构，计算机作为客户端，探测器作为服务器，接收计算机命令并响应，采用 UDP 通讯协议。

探测器上电后，通过固定 IP 192.168.0.2 : 5000 发出广播，广播信息如表三所示：

表三 广播信息

探测器 MAC1	探测器 MAC2	探测器 MAC3	探测器 MAC4
探测器 MAC5	探测器 MAC6	探测器 IP1	探测器 IP2
探测器 IP3	探测器 IP4	主机 IP1	主机 IP2
主机 IP3	主机 IP4	命令端口 1	命令端口 2
图像端口 1	图像端口 2	出厂序列号 1	出厂序列号 2
出厂序列号 3	出厂序列号 4	出厂序列号 5	出厂序列号 6
出厂序列号 7	出厂序列号 8	客户序列号 1	客户序列号 2
客户序列号 3	客户序列号 4	客户序列号 5	客户序列号 6
客户序列号 7	客户序列号 8	预留字段 1	预留字段 2
预留字段 3	预留字段 4		

主机通过广播信息获得探测器 IP 地址，向探测器发送 ARP 请求命令，探测器同时可获得主机地址及端口号，从而建立点对点的 UDP 通讯。

5.2 点对点通讯协议

探测器默认参数：

IP 地址：192.168.0.2

命令端口：3000

数据端口：4001

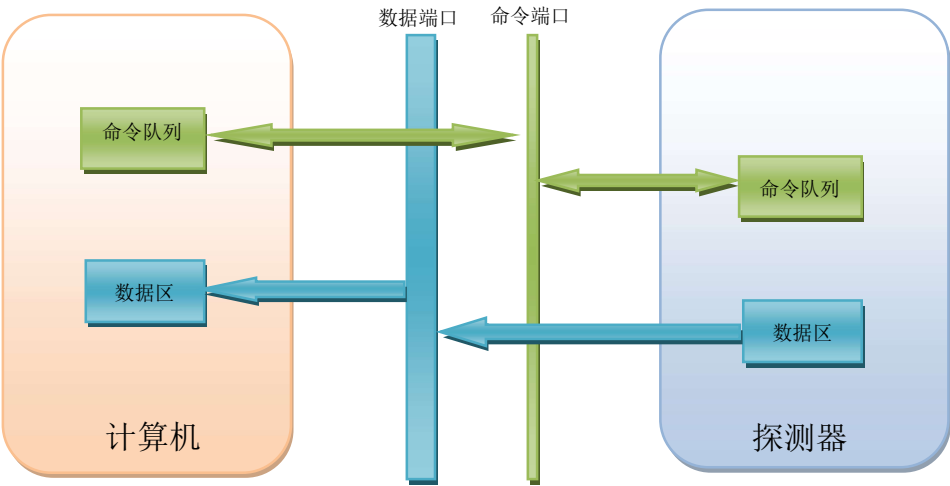


图 15 通讯示意图

5.2.1 命令端口

命令端口用来将命令传达至探测器，探测器对计算机命令进行响应，并返回结果。

所有指令均在 ASCII 中并且遵循以下规则：指令主体的第一特征是“[”，此特征被定义为 STX（正文开始）。指令主体的最后特征是“]”，此特征被定义为 ETX（文本结束）。指令关键词是 ASCII 格式 A 到 Z 的大写字母。“,”用来隔开指令键与参数。对于任何数字参数，都以其十六进制数字的 ASCII 码被发送。比如说,需要发送参数 4000,发送到探测器的物理数据是 0FA0。指令“[ST,W,0FA0]”是用来设定探测器的积分时间为 4ms。具体命令参见附录 2（命令列表及说明）。

5.2.2 图像数据端口

数据端口是用来接收探测器图像数据的，这些数据是以数据包格式从探测器传输到计算机，采取 UDP 协议。



图 16 数据包格式

包标志:

长度 2 字节



包信息:

长度 2 字节



包信息字中，高 12 位代表数据包内图像数据的字节数；低 4 位代表每行图像打包分装的数据包索引最大值。

比如：包信息字为 HEX 1603 ，高 12 位 HEX 160 代表此包中图像数据有 HEX 160（DEC 352）个字节，低 4 位 HEX 3 代表每行的包索引最大值为 3，即每行数据包索引为 0-3，共 4 包。可据此信息获得每行数据的大小。

行信息:

长度 2 字节



行编号 HEX 000- HEX FFF

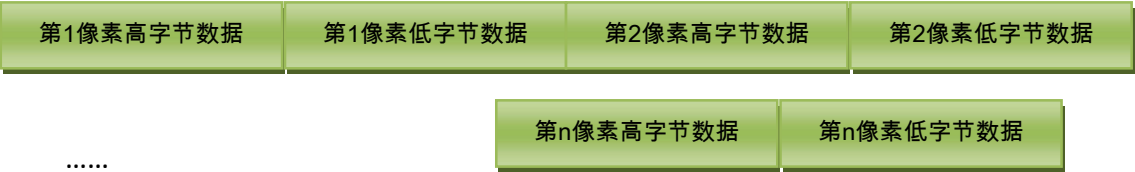
每行包编号 HEX 0 – HEX F

比如：行信息 HEX 46C3，高 12 位 HEX 46C 代表本包数据的行编号为 HEX 46C，低 4 位 HEX 3 代表本包数据是 HEX 46C 行的 第 4 包数据，0 – F 对应第 1 至 16 包。

此信息可用于判断丢行及行数据缺失情况。

数据:

长度为包信息中指明的长度



5.3 像素数据与图像映射

图像数据与卡的连接以及卡类型直接相关，下面从卡的连接以及卡类型这两个方面进行说明。

卡数设置：

卡数设置对于探测器属于逻辑上(与卡类型无关)的划分，逻辑上将 64 个像素(目前应中用最小的划分单位)作为 1 块卡看待，比如设置命令[DN,W,0,10]，设置 P1 通道上接入了 HEX10 (DEC 16) 块卡，也即 $64 \times 16 = 1024$ 个像素。

注意，[DN,W,X,XX]指令中，X 为通道号，0~3，一位数字；XX 为卡数量，两位十六进制数。

在数据包中，通道 P1 数据在先，依次 P2，P3，P4，某通道卡数为 0 时，越过进入下一通道。

卡类型：

D-DBR7000 并不获得所连接卡是单能还是双能的信息，如果连接了 1 块 DS16A64SE 卡（64 像素），对于 D-DBR7000 意味着需要设置卡数为 1；如果连接了 1 块 DS16A64DE 卡（128 像素），对于 D-DBR7000 意味着需要设置卡数为 2。

示例：信号采集接口 P1 连接 6 块双能传感器卡 **DS16A64DE**，信号采集接口 P2 连接 5 块单能传感器卡 **DS16A64SE**。P3，P4 未连接探测器卡。则需要进行如下设置：

[DN,W,0,0C] P1 通道对应 6 块双能，也即 $128 \times 6 = 768$ 个像素，逻辑上卡数为 $768/64=12$ 。

[DN,W,1,05] P2 通道对应 5 块单能，也即 $64 \times 5 = 320$ 个像素，逻辑上卡数为 5。

[DN,W,2,00] P3 无卡接入。

[DN,W,3,00] P4 无卡接入。

像素数据为：



图 17 像素数据排列

像素数据中，P1 连接 6 块双能卡，共计 768 个像素；P2 连接 5 块单能卡，共计 320 个像素，探测器合计像素数为 1088 个。

对于双能卡每块 128 个像素数据，64 个低能像素数据在前，64 个高能像素数据在后。示例中，

像素数据中第 1 像素数据至第 64 像素数据为第 1 块卡的低能数据，第 65 像素数据至第 128 像素数据为第 1 块卡的高能数据，以此类推，第 641 像素数据至第 704 像素数据为第 6 块卡的低能数据，第 705 像素数据至第 768 像素数据为第 6 块卡的高能数据，按此排列，合成伪彩图像。

5.4 编程实例

本节简单介绍演示程序的编程，详细请参见设备配套的软件及源码。

5.4.1 发送命令

```
private void sendCmd()
{
    IPEndPoint ep = new IPEndPoint(xImageIP, cmdPortUDP);
    IPEndPoint senderIP = new IPEndPoint(IPAddress.Any, cmdPortUDP);
    EndPoint Remote = (EndPoint) senderIP;
    while (sctUDP.Available > 0)
    {
        bytes = sctUDP.ReceiveFrom(revBytes, ref Remote);
        sctUDP.SendTo(sendBytes, sendBytes.Length, SocketFlags.None, ep);
        sctUDP.SetSocketOption(SocketOptionLevel.Socket, SocketOptionName.ReceiveTimeout, 100);
        Thread.Sleep(20);
        if (sctUDP.Available != 0)
        {
            bytes = sctUDP.ReceiveFrom(revBytes, ref Remote);
        }
        else
        {
            return;
        }
        strRev = Encoding.ASCII.GetString(revBytes, 0, bytes);
    }
}
```

5.4.2 绘制检测结果

```
private void creatDataImage(byte[] addLine, int iType)
{
    iCurrentScanLineNo++;
    if (iCurrentScanLineNo >= iHeight)
    {
        iCurrentScanLineNo--;
        Array.Copy(iCurrentShow, iWidth, iCurrentShow, 0, iWidth * (iHeight - 1));
        Array.Copy(bDrawData, iWidth, bDrawData, 0, iWidth * (iHeight - 1));
    }
    int copyPos = iCurrentScanLineNo * iWidth;
    int ib = 0;
    for (int i = 0; i < iWidth; i++)
    {
        iCurrentShow[iHeight - 1, i] = (int)(addLine[6 + i * 2] * 256 + addLine[7 + i * 2]);
    }
}
```

```

        ib = (int)((addLine[6 + i * 2] - iLow256) * sTimes256);
        if (ib < 0) ib = 0;
        if (ib > 255) ib = 255;
        bDrawData[copyPos + i] = (byte)ib;
    }
    iCurrentLine += 1;
    if (iCurrentLine == iHeight)
    {
        iCurrentLine = 0;
        bShow = !bShow;
    }
}

private void timerDrawData_Tick(object sender, EventArgs e)
{
    if (!bRev)
        return;

    drawData();
}

private void drawData()
{
    Rectangle rect = new Rectangle(0, 0, iWidth, iHeight);
    bmpData = bbB.LockBits(rect, System.Drawing.Imaging.ImageLockMode.ReadWrite, bbB.PixelFormat);
    IntPtr ptr = bmpData.Scan0;
    System.Runtime.InteropServices.Marshal.Copy(bDrawData, 0, ptr, bDrawData.Length);
    bbB.UnlockBits(bmpData);
    Graphics graphics = pictureBox1.CreateGraphics();
    graphics.DrawImage(bbB, 0, 0);
    graphics.Dispose();
}

```

5.4.3 接收检测数据

```

private void receiveAndShow()
{
    byte[] buffer = null;
    button3.Focus();
    int iRevBs = 1600 * 320;
    buffer = new byte[iRevBs];
    totalFrames = 0;
    totalLines = 0;
    badFrames = 0;
}

```

```

bRev = true;
kkkk = 0;
if (bDecConnected == false)
    return;
bi = new byte[3];
bi[0] = 0;
bi[1] = 1;
bi[2] = 6;
bbB = new Bitmap(iWidth, iHeight, System.Drawing.Imaging.PixelFormat.Format8bppIndexed);
pictureBox1.Width = iWidth;
System.Drawing.Imaging.ColorPalette cp = bbB.Palette;
for (int ci = 0; ci < cp.Entries.Length; ci++)
{
    cp.Entries[ci] = Color.FromArgb(ci, ci, ci);
}
bbB.Palette = cp;
iCurrentLine = 0;
bSaveTest = true;
EndPoint senderIP = new EndPoint(IPAddress.Any, imgPortUDP);
EndPoint Remote = (EndPoint)senderIP;
sctImgUDP.ReceiveBufferSize = 500000;
MemoryStream msLine = new MemoryStream();
int currentLine = 0;
int separatLine = 0;
int currentPack = 0;
iBadPack = 0;
sBadPack = new string[iRecLog];
bDrawData = new byte[iWidth * iHeight];
int iTimeout = 0;
iCurrentScanLineNo = 0;
timerDrawData.Start();
while (bRev)
{
    if (sctImgUDP.Available != 0)
    {
        iTimeout = 0;
        byte[] revBytes = new byte[16000];
        int recv = sctImgUDP.ReceiveFrom(revBytes, ref Remote);
        if (sBadPack.Length != iRecLog)
        {
            sBadPack = new string[iRecLog];
            iBadPack = 0;
        }
        if (revBytes[0] != 255 || revBytes[1] != 255)

```

```

{
    badFrames++;
    msLine.Position = 0;
    if (bRecordLogFile)
    {
        int iPx = (revBytes[2] * 256 + revBytes[3]) / 16;
        int iPk = revBytes[3] & 0x0f;
        iPk++;
        int iLn = (revBytes[4] * 256 + revBytes[5]) / 16;
        int iPn = revBytes[5] & 0x0f;
        sBadPack[iBadPack] = badFrames.ToString() + "\t" + DateTime.Now.ToString() + "\t" +
"TYPE:0" + "\t" +
        revBytes[0].ToString() + "\t" + revBytes[1].ToString() + "\t" + iPx.ToString() +
"\t" +
        iPk.ToString() + "\t" + iLn.ToString() + "\t" + iPn.ToString() + "\t" +
currentPack.ToString();

        iBadPack++;
        if (iBadPack > iRecLog || iBadPack == iRecLog)
        {
            iBadPack = 0;
            Thread tRecLog = new Thread(recBadPack);
            tRecLog.Start();
        }
    }
    currentPack = 0;
    continue;
}

int iPack = revBytes[5] & 0x0f;
if (iPack > 0 && (iPack != currentPack + 1))
{
    badFrames++;
    msLine.Position = 0;
    if (bRecordLogFile)
    {
        int iPx = (revBytes[2] * 256 + revBytes[3]) / 16;
        int iPk = revBytes[3] & 0x0f;
        iPk++;
        int iLn = (revBytes[4] * 256 + revBytes[5]) / 16;
        int iPn = revBytes[5] & 0x0f;
        sBadPack[iBadPack] = badFrames.ToString() + "\t" + DateTime.Now.ToString() + "\t" +
"TYPE:1" + "\t" +
        revBytes[0].ToString() + "\t" + revBytes[1].ToString() + "\t" + iPx.ToString() +
"\t" +
        iPk.ToString() + "\t" + iLn.ToString() + "\t" + iPn.ToString() + "\t" +

```

```

currentPack.ToString();

        iBadPack++;
        if (iBadPack > iRecLog || iBadPack == iRecLog)
        {
            iBadPack = 0;
            Thread tRecLog = new Thread(recBadPack);
            tRecLog.Start();
        }
    }
    currentPack = 0;
    continue;
}

int iPixels = (revBytes[2] * 256 + revBytes[3]) / 32;
separatLine = revBytes[3] & 0x0f;
separatLine++;
iPixels = iPixels * separatLine;           // pixels in a line
if (iPixels != iWidth)                     // when card number(pixels per line) changed
{
    iWidth = iPixels;
    iCardNum = iWidth / 64;
    iAverageline = new int[iWidth];
    iSumLine = new int[iWidth];
    iCurrentShow1 = new int[iHeight, iWidth];
    iCurrentShow2 = new int[iHeight, iWidth];
    iCurrentShow = new int[iHeight, iWidth];
    iCurrentLine = 0;
    iCurrentScanLineNo = 0;
    bDrawData = new byte[iWidth * iHeight];
    bbB = new Bitmap(iWidth, iHeight, System.Drawing.Imaging.PixelFormat.Format8bppIndexed);
    bbB.Palette = cp;
    pictureBox1.Width = iWidth;
    bShow = true;
    msLine.Position = 0;
    drawCardNum(iWidth);
}

if (iPack == 0) currentPack = 0;
currentPack = iPack;
if (currentPack == 0)
{
    msLine.Position = 0;
    byte[] bHead = new byte[6];
    bHead[0] = 0;
    bHead[1] = bi[0];
    bHead[2] = 6;

```

```

        msLine.Write(bHead, 0, 6);
    }
    msLine.Write(revBytes, 6, recv - 6);
    if (currentPack == separatLine - 1)    //get all packs in a line
    {
        totalLines++;
        msLine.Position = 0;
        msLine.Read(buffer, 0, iRevBs);
        creatDataImage(buffer, 2);
        totalFrames = totalLines / iHeight;
        toolStripStatusLabel1.Text = "Total Frames:" + totalFrames + ";    bad packs:" + badFrames;
    }
}
else
{
    iTimeout++;
    if (iTimeout > 1000000)
    {
        bRev = false;
        stopReceive();
        MessageBox.Show("receive data time out! cheak network please...");
        iTimeout = 0;
    }
}
Application.DoEvents();
}
msLine.Close();
}

```

第六章 动态链接库CHIGSDetector.dll调用及开发

6.1 简介

动态链接库文件 CHIGSDetector.dll，专用于主机客户端软件调用连接 D-DBR7000 板卡，与板卡通讯、采集板卡数据使用。

6.2 数据结构及方法说明

6.2.1 自定义结构 HIGSCollectorPara，用于保存探测器卡参数

```
struct HIGSCollectorPara
{
//HIGS 相关参数定义
    char ipRemoteIP[50]; //探测器 IP 地址字符串
    char ipLocalIP[50];  //客户端本地 IP 地址字符串
    unsigned char ipRemoteChange[50]; //更改探测器 IP 地址及端口号字节流
    int iImagePort; // 探测器图形端口
    int iCmdPort; // 探测器指令端口
    int iCardType; // DBR&DAS4000 型系统类型值为 2
    int iIntTime; // 探测器积分时间
    int iCardNum; // 探测器卡总数量
    int iCardInC1; // 探测器通道 1 卡数量
    int iCardInC2; // 探测器通道 2 卡数量
    int iCardInC3; // 探测器通道 3 卡数量
    int iCardInC4; // 探测器通道 4 卡数量
};
```

6.2.2 方法清单

① 初始化 bool Initialize(const CollectorPara* pPara)

将结构中的数据上传至探测器卡，并且尝试网络连接探测器卡。设置及连接成功则返回 true，否

则返回 false，可调用 GetLastError() 查看错误代码，参考附录 4 获得错误原因。

② 清理毁数 bool UnInitialize()

清理内存数据、清空缓存区

③ bool Start()

启动扫描。成功后返回 true，之后可以通过调用 SetReportFunc 回调函数，实现扫描数据的读取。

④ bool Stop()

停止扫描，成功后返回 true。

⑤ bool ReadData(*/*OUT*/ unsigned char* pData, /*IN*/ unsigned dwLength, /*OUT*/ unsigned& dwIndex*)

接收扫描数据。输入参数 dwLength 代表索取的数据大小加 4 字节，例如每行扫描线为 512 个像素，每像素 2 个字节，则 dwLength 的值为 $512 \times 2 + 4 = 1028$ 字节。

*pData 是输出的扫描数据，数据格式如下图：



⑥ unsigned GetLastError()

返回当前错误代码值，参考附录 4 获得错误原因。

⑦ bool SetPara(unsigned dwType, unsigned char* dwValue, int iDataLen)

设置探测器卡参数。dwType 为设置参数代码，例如设置探测器 IP 地址及端口号，dwType=5001；*dwValue 为设置值的字节流；iDataLen 是 dwValue 的长度。参考附录 5 设置参数列表。

⑧ unsigned GetPara(unsigned dwType)

读取探测器参数。dwType 为输入需要读取的参数编号，例如 dwType=107，则返回每像素的字节数。

⑨ 查看日志

在动态链接库的同一文件夹下，有“log.txt”日志文件，其中保存着调用动态链接库的实时信息，方便查阅。

6.3 调用方式

使用 MFC 编写调用本动态链接库软件时，需将 CHIGSDetector.dll、IDataCollector.h 文件拷贝到 MFC 编写的可执行文件同一文件目录中。在头文件中加入如下代码：

```
#include "../CHIGSDetector/IDataCollector.h"

typedef IDataCollector *( *pCreateDataCollector)(unsigned);

typedef bool *( *pDestroyDataCollector)(IDataCollector *);

private:

    IDataCollector *pIDataCollector;

    HMODULE hHIGSDetectorModule;
```

6.3.1 工作流程

① 创建实例：

```
pCreateDataCollector    pDC=(pCreateDataCollector)GetProcAddress(hHIGSDetectorModule,
"CreateDataCollector");

    pIDataCollector=pDC(0);
```

② 探测器相关参数赋值：

```
CollectorPara para;

para.iCardType=CARDTYPE_UDP;

para.iCmdPort=3000;

para.iImagePort=4001;

para.iCardInC1=20;

para.iCardInC2=20;

para.iCardInC3=20;

para.iCardInC4=20;

para.iIntTime=3000;

.....
```

③ 探测器初始化:

```
pIDataCollector->Initialize(&para);
```

④ 启动扫描:

```
pIDataCollector->Start();
```

⑤ 停止扫描:

```
pIDataCollector->Stop();
```

⑥ 探测器清理:

```
pIDataCollector->UnInitialize();
```

```
pDestroyDataCollector pDDC=(pDestroyDataCollector)GetProcAddress(hHIGSDetectorModule,
"DestroyDataCollector");
```

```
pDestroyDataCollector(pIDataCollector);
```

⑦ 通过回调的方式接受扫描数据

```
this->pIDataCollector->SetReportFunc(ReportFuncCallback);
```

```
void ReportFuncCallback()
```

```
{
    while(pIDataCollector->ReadData(pLine, iBuffLen, dwIndex))
    {
        int lineNo;
        iCurrentScanLineNo++;
        if(iCurrentScanLineNo>=ScanImageHeight)
        {
            iCurrentScanLineNo=iCurrentScanLineNo-1;
            memcpy(pScanImageBytes, pScanImageBytes+ScanImageWidth,
                ScanImageWidth*(ScanImageHeight-1));
        }
        unsigned char *pLinePos=pScanImageBytes+(iCurrentScanLineNo*ScanImageWidth);
        for(int i=0;i<ScanImageWidth;i++)
        {
            pLinePos[i]=pLine[sizeof(int)+i*PixelSize*i];
        }
    }
}
```

6.3.2 工作流程图

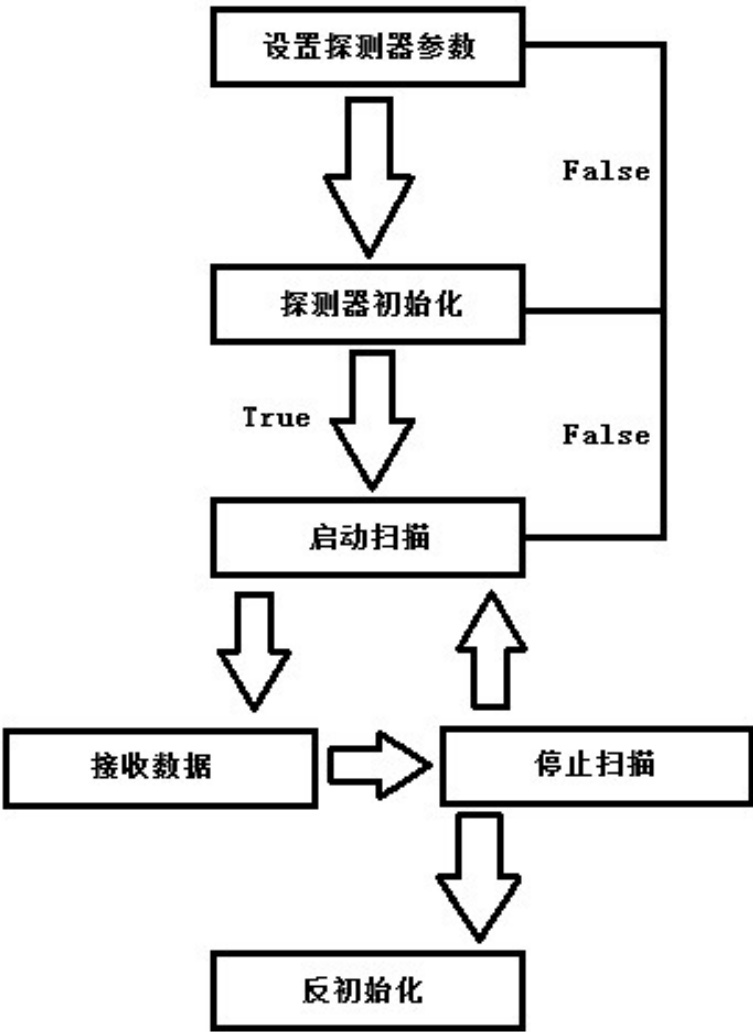


图 18

附录

附录 1 D-DBR 机械安装尺寸

“台”封面图																																																																									
零件代号																																																																									
催(通)用件登记																																																																									
旧底图总号																																																																									
底图总号																																																																									
签字																																																																									
日期																																																																									
<table border="1"> <tr> <td rowspan="2">标记</td> <td>处数</td> <td>分区</td> <td>更改文件号</td> <td>签名</td> <td>年月日</td> <td rowspan="2">阶段</td> <td rowspan="2">标记</td> <td rowspan="2">质量</td> <td rowspan="2">比例</td> <td rowspan="2">“图样名称”</td> </tr> <tr> <td>设计</td> <td></td> <td></td> <td>标准化</td> <td></td> <td></td> <td></td> <td>0.000</td> <td>2:1</td> <td rowspan="2">“图样代号”</td> </tr> <tr> <td>校核</td> <td></td> <td></td> <td></td> <td>工艺</td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> </tr> <tr> <td>主管设计</td> <td></td> <td></td> <td></td> <td>审核</td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> </tr> <tr> <td></td> <td></td> <td></td> <td></td> <td>批准</td> <td></td> <td></td> <td></td> <td></td> <td></td> <td></td> </tr> <tr> <td colspan="6">共1张 第1张 版本</td> <td colspan="4">替代</td> </tr> </table>										标记	处数	分区	更改文件号	签名	年月日	阶段	标记	质量	比例	“图样名称”	设计			标准化				0.000	2:1	“图样代号”	校核				工艺							主管设计				审核											批准							共1张 第1张 版本						替代			
标记	处数	分区	更改文件号	签名	年月日	阶段	标记	质量	比例		“图样名称”																																																														
	设计			标准化									0.000	2:1	“图样代号”																																																										
校核				工艺																																																																					
主管设计				审核																																																																					
				批准																																																																					
共1张 第1张 版本						替代																																																																			

附录 3 D-DBR 系列主板命令列表及说明

Command	Parameter	Range	Example	Comment	Note
TM	W, Pattern Mode	0 - 1	[TM,W,1]	1: Enable data pattern 0: Disable data pattern	
SF	W, 1/0		[SF,W,1]	1: Start grabbing 0: Stop grabbing	
ST	W/R, Integration Time	125 - 20000	[ST,W,0FA0]	Integration time unit is us. And the input parameter format is in HEX	1
SS	W/R, CardNo. Sensitivity	Card No.1-20 Sensitivity: 0- 62	[SS,W,01,52]	Set the sensitivity level. the input parameter format is in ASIIC	
FT	W/R, 1/0		[FT,W,1]	0: Disable frame trigger 1: external frame trigger, High level 2: external frame trigger, pos edge	
LT	W,0/1/2		[LT,W,1]	0: Disable line trigger 1: Ext line trigger and integration time 2: Ext line trigger, fix integration time	
DN	W, Channal No. Card Num.	1 – 20	[DN,W,0,08]	Set the cards number for the first channel	
DT	W, Detector type	0,1,2	[DT,W,1]	0: 32pixels 1: 64pixels 2: 128pixels 3: 256pixels	
SV	W,1	1	[SV,W,1]	1: Save config	2
SE	W, 0/1	0-1	[SE,W,1]	0: High byte first default 1: Low byte first	
SN	W, Detector IP, CMD port, IMG port	1	[SN,W,xxxxxxxx]	IP 4 bytes CMD port 2Bytes IMG port 2Bytess	
CC	W, customer code	8 bytes	[CC,W,12345678]	Write customer code	

[TM,W,0] 测试模式设置命令

用于测试 D-DBR7000 主板数据传输功能，也可为应用程序开发提供帮助。

[TM,W,0] 设置为真实数据，数据来源于探测器。

[TM,W,1] 设置为测试模式，数据为预置数据，起始为 514，后续数据按 257 递增。

[SF,W,0] 扫描启停控制

用于控制探测器扫描采集的启停，在进行卡数设置、积分时间变更时先进行停止扫描操作。

[SF,W,0] 停止扫描采集。

[SF,W,1] 启动扫描采集。

[ST,W,xxxx] 设置积分时间

用于积分时间设置，积分时间单位为微秒，比如设置积分时间 4ms，需要设置参数为 4000，换算成 16 进制为 0FA0（需要 4 位十六进制），所以设置命令为[ST,W,0FA0]。

最大允许积分时间限定为 20ms，最短积分时间取决于连接的卡数多少，当设置的积分时间小于探测器能提供的最小积分时间时，命令会返回该可设置的最小积分时间，并按此进行了内部设置。

[SS,x,xx,xx] 设置探测器敏感度

设置探测器内部积分电容。63 级增益可调，积分电容可调范围 0.375pF — 23.6pF，对应设置值为 0—62。

[FT,W,x] 外部帧触发设置

外部帧触发常用于采集图像时同步。

[FT,W,0] 内部自动触发。

[FT,W,1] 外部帧触发，高电平有效。

[FT,W,2] 外部帧触发，上升沿有效。

在使用外部帧触发时，在停止扫描的状态下，发送[FT,W,x]命令，然后启动扫描，D-DBR7000 将检测外部帧触发信号，待得到正确触发信号后开始采集数据，数据采集的停止由用户自行设置。在扫描中外部信号再次触发无效，只有待数据采集停止后方可再次进行设置。

[LT,W,x] 外部线触发设置

外部线触发，也称为行频，即每一个脉冲对应探测器的一次扫描，一行的数据。脉冲的周期对应行数据的出现周期。

[LT,W,0] 内部线触发。

[LT,W,1] 外部线触发，脉冲周期对应积分周期。

[LT,W,2] 外部线触发，脉冲周期对应行数据出现的周期，积分时间采用设置的时间，不随线触发周期变动。线触发周期应该大于内部积分时间。

[DN,W,x,yy] 设置通道探测器卡数

设置通道上连接的卡数量。

在 D-DBR7000 中逻辑上每块卡具有 64 个像素，每块单能卡像素为 64 个，每块双能卡像素为 64*2 个，所以当连接双能卡时，逻辑上的卡块数设置需要翻倍，例如，第一链路连接了 8 块双能卡，则需要设置

[DN,W,0,10], 10 为十六进制数, 换算为十进制为 16 (8*2)。如果第一链路连接了 8 块单能卡, 则可直接设置[DN,W,0,8]。

D-DBR7000 支持 4 路通道。

[DN, W, 0, xx] 设置通道 1 上连接的卡数;

[DN, W, 1, xx] 设置通道 2 上连接的卡数;

[DN, W, 2, xx] 设置通道 3 上连接的卡数;

[DN, W, 3, xx] 设置通道 4 上连接的卡数。

[DT,W,x] 设置探测器类型

D-DBR7000 可支持多种型号的 DAS 卡, 每块卡中像素数目。

[SV,W,1] 保存参数配置

执行此命令, 系统会自动保存当前每通道卡数设置、卡类型以及积分时间, 再次启动后直接调用此参数为默认参数。

[SN,W,xxxxxxxx] 设置网络通讯参数

执行此命令将设置探测器的 IP 地址, 命令通道端口, 图像数据端口, 重新上电后参数生效。

[CC,W,xxxxxxxx] 设置客户产品编码

系统预留 8 个字节, 可用于存储客户信息, 此信息在广播时会发出。

附录 4 动态链接库错误代码表

错误代码	描述	原因分析
0	无错误	
100	连接探测器错误	检查网络参数设置
199	网络超时	检查网络参数设置
200	发送测试指令错误	检查网络参数设置
201	启动扫描错误	启动扫描指令探测器返回值错误
202	发送启动扫描指令错误	检查网络参数设置
203	停止扫描指令发送错误	检查网络参数设置
204	探测器卡数量设置错误	设置的探测器卡数量值超出有效范围
205	探测器积分时间设置错误	设置的探测器卡积分时间值超出有效范围
206	更改探测器 IP 地址错误	更改的 IP 地址或端口号字节流不在有效范围之内
300	获取探测器参数错误	读取参数类型不在有效列表内

附录 5 设置参数列表

设置参数代码	指令含义	数据格式
5001	设置探测器 IP 地址及端口号	IP 地址 指令端口 图形端口